

Aggregate Functions

Authored by: Arpit Raj, Lnmiit jaipur

Chapter 1 — Introduction to Aggregate Functions

1.1 What are Aggregate Functions?

Definition

Why aggregation is important

Single-value output from multiple rows

```
SELECT SUM(salary)
FROM Employee;
```

SQL looks at all four salary values

- 50000
- 70000
- 65000
- 80000

and produces only one value

265000

Function	Purpose
COUNT()	Count rows
SUM()	Add values
AVG()	Average
MIN()	Smallest value
MAX()	Largest value

Almost every SQL database supports these.

Instagram

Total followers

```
SELECT COUNT(*)  
FROM Followers;
```

Netflix

Average movie rating

```
SELECT AVG(rating)  
FROM Movies;
```

1.2 Aggregate Functions vs Scalar Functions

Row-wise operations

Group-wise operations

A scalar function

UPPER(name)

looks at
One row



One output

Aggregate function

SUM(salary)

looks at
Entire table



One output

This difference is extremely important.

1.3 Logical Query Execution

Where aggregate functions execute in the SQL pipeline.

Aggregate functions are not executed immediately.

They execute after WHERE has filtered the rows.

This is extremely important.

Why Can't Aggregate Functions Be Used in WHERE?

Many beginners write

```
SELECT *  
FROM Employee  
WHERE AVG(salary)>50000;
```

This produces an error.

Why?

Because **WHERE** executes **BEFORE AVG()**

At the time WHERE is running, AVG hasn't even been calculated yet.

1.4 NULL Handling

Ignoring NULL values

Exceptions

Empty result sets

Almost every aggregate function ignores NULL values. That means NULL is skipped during calculations.

1.5 Aggregate Function Categories

Counting

Summation

Averaging

Extremes

Statistical aggregates

Chapter 2 — COUNT()

2.1 Introduction

Counting rows.

2.2 Syntax

2.3 COUNT(*)

Count all rows.

COUNT(*) This is the exception. COUNT(*) counts rows. It does not care whether columns are NULL.

2.4 COUNT(column)

Ignore NULL values.

COUNT(column) This is different.

Syntax

```
SELECT COUNT(email)
FROM Employee;
```

Here, COUNT only counts **Non-NULL** email values.

2.5 COUNT(DISTINCT)

Unique values only.

What is COUNT(DISTINCT)?

COUNT(DISTINCT) counts the number of unique (different) non-NULL values in a column

```
SELECT COUNT(DISTINCT city)
FROM Student;
```

SQL first looks at all cities

- Delhi
- Mumbai
- Delhi
- Kolkata
- Mumbai

Then removes duplicates

- Delhi
- Mumbai
- Kolkata

Then counts

3

COUNT(DISTINCT Multiple Columns)

Some databases support

```
SELECT COUNT(DISTINCT city, state)
FROM Customers;
```

Meaning

Count unique (city, state) pairs.

Example

City	State
Delhi	Delhi
Delhi	Delhi
Delhi	Haryana
Mumbai	Maharashtra

Unique pairs

- (Delhi, Delhi)
- (Delhi, Haryana)
- (Mumbai, Maharashtra)

Answer

3

2.6 COUNT with Expressions

```
salary > 50000
```

Table

Salary	Expression
40000	FALSE
60000	TRUE
70000	TRUE

Query

```
SELECT COUNT(salary > 50000)
FROM Employee;
```

Important Vendor Behavior

This is not portable SQL.

- MySQL treats TRUE and FALSE as non-NULL values (internally 1 and 0), so this query returns 3, because every row produces either TRUE or FALSE—not NULL.
- PostgreSQL also evaluates the boolean expression for every non-NULL salary, so COUNT(expression) counts all non-NULL boolean results.

If salary itself is NULL, then salary > 50000 evaluates to NULL, and that row is not counted. Therefore, do not use COUNT(boolean_expression) when you want to count matching rows

Correct Way

Use

```
SELECT COUNT(*)  
FROM Employee  
WHERE salary > 50000;
```

COUNT(CASE...)

This is one of the most useful SQL techniques.

Example

Name	Department	Evaluation	COUNT
Aadya	IT	1	1
Arpit	HR	NULL	
Riya	IT	1	1
Kabir	Finance	NULL	

Query

```
SELECT COUNT(  
    CASE  
        WHEN department='IT'  
        THEN 1  
    END  
)  
FROM Employee;
```

Answer

2

MySQL Shortcut

Instead of CASE, MySQL developers often write

```
COUNT(IF(department='IT',1,NULL))
```

2.7 COUNT on Empty Tables

2.8 Performance Considerations

- Indexes
- Storage engine behavior

2.9 Real-World Examples

- Orders
- Students
- Employees
- Products

2.10 Practice Problems

Summary

Function	Empty Table
COUNT(*)	0
COUNT(column)	0
COUNT(DISTINCT)	0
SUM()	NULL
AVG()	NULL
MIN()	NULL
MAX()	NULL

Chapter 3 — SUM()

3.1 Introduction

Adding numeric values.

3.2 Syntax

3.3 SUM with Integers

3.4 SUM with Decimals

3.5 SUM(DISTINCT)

3.6 NULL Behavior

Empty table

```
SELECT SUM(salary)
FROM Employee;
```

Output

NULL

Unlike COUNT(), SUM() returns NULL when there are no non-NULL values to add.

Using COALESCE

```
SELECT COALESCE(SUM(salary),0)
FROM Employee;
```

Output

0

3.7 Financial Examples

Revenue

Expenses

Profit

3.8 Overflow Considerations

3.9 Practice Problems

Chapter 4 — AVG()

4.1 Introduction

Average values.

4.2 Syntax

4.3 Integer Average

4.4 Decimal Average

4.5 AVG(DISTINCT)

4.6 NULL Handling

4.7 Rounding Results

AVG often returns long decimal values.

Example

200.246666666

Use ROUND

```
SELECT ROUND(AVG(price),2)
FROM Product;
```

4.8 Financial & Analytics Examples

4.9 Practice Problems

Chapter 5 — MIN()

5.1 Introduction

Smallest value.

5.2 Syntax

Basic syntax

```
SELECT MIN(column_name)
FROM table_name;
```

With WHERE

```
SELECT MIN(salary)
FROM Employee
WHERE department='IT';
```

5.3 Numeric Values

5.4 Strings

Alphabetical order

MIN compares strings lexicographically (dictionary order).

5.5 Dates

Earliest date

Dates are compared chronologically.

5.6 NULL Handling

5.7 Index Optimization

```
SELECT MIN(id)
FROM Employee;
```

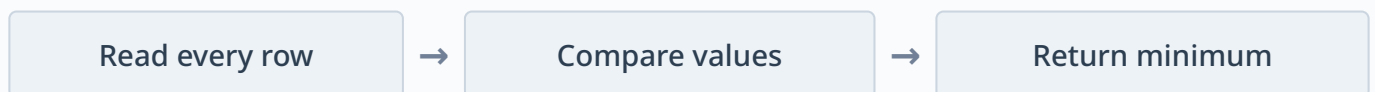
If an index exists, the database can often find the first (smallest) indexed value without scanning the whole table.

Similarly,

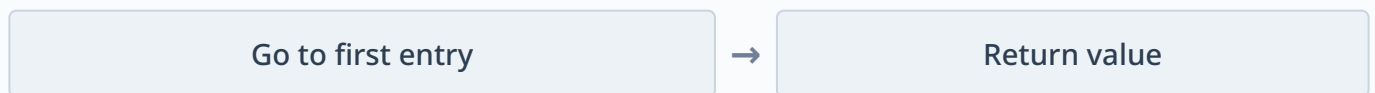
```
SELECT MIN(salary)
FROM Employee;
```

can also benefit from an index on salary.

Without an index



With an index



Much faster.

5.8 Practice Problems

Chapter 6 — MAX()

6.1 Introduction

Largest value.

6.2 Syntax

```
SELECT MAX(column_name)
FROM table_name;
```

With WHERE

```
SELECT MAX(salary)
FROM Employee
WHERE department='IT';
```

6.3 Numeric Values

6.4 Strings

Lexicographical order

6.5 Dates

Latest date

6.6 NULL Handling

6.7 Index Optimization

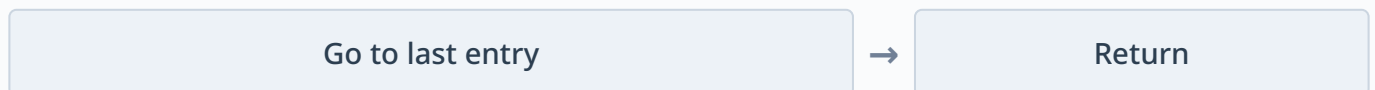
```
SELECT MAX(salary)
FROM Employee;
```

With an index, the database can often retrieve the last (largest) indexed value instead of scanning the entire table.

Without an index



With index



6.8 Practice Problems